

LLM-Based Multi-Agent Mobile QA Automation: Architecture and Experimental Evaluation

Soham Sarkar, QualGent Research Intern Candidate, December 2025

Abstract—This work presents a novel multi-agent system leveraging large language models (LLMs) for autonomous mobile quality assurance automation on real Android devices. We developed a three-tier agent architecture combining planning, execution, and supervision capabilities, integrated with Android Debug Bridge (ADB) for device control. The system was evaluated using the Obsidian Android application with four comprehensive test cases covering onboarding workflows, note creation, and negative validation scenarios. Our implementation demonstrates LLM-based agents’ capabilities in understanding UI state, generating action sequences, and managing complex multi-step workflows. The experimental evaluation on a physical Pixel 6 Pro device reveals both the potential and current limitations of autonomous agentic systems in mobile testing contexts. We document the complete architecture, provide quantitative performance metrics, and share insights on effective LLM integration strategies for mobile automation.

Index Terms—Mobile Quality Assurance, Large Language Models, Multi-Agent Systems, Android Automation, Agentic AI, Test Automation, UI Testing, Agent Architecture

I. INTRODUCTION

Mobile application testing presents unique challenges due to dynamic user interfaces, diverse device characteristics, and complex user interaction patterns. Recent advances in large language models have opened new possibilities for intelligent automation systems capable of understanding application context and autonomously executing test workflows. This research explores the application of LLM-based multi-agent systems to mobile QA automation, investigating how language models can reason about UI state, plan appropriate actions, and coordinate complex testing workflows. We developed a complete agent-based architecture that integrates planning, execution, and supervision capabilities, demonstrating how modern AI can be applied to practical testing challenges. Our experimental platform uses the Obsidian Android application running on a physical Pixel 6 Pro device, providing authentic conditions for evaluating agent behavior. Through comprehensive testing scenarios, we examine the agents’ ability to handle onboarding flows, create and manipulate content, and correctly identify expected failure conditions.

The research contributions of this work include:

- A novel multi-agent architecture integrating LLM-based planning with mobile automation capabilities
- Quantitative evaluation of LLM agent performance in real-world mobile testing scenarios
- Analysis of agent behavior patterns, including successful strategies and observed limitations
- Video demonstration and open-source implementation for reproducibility

- Design insights for building effective LLM-based mobile testing systems

II. ENVIRONMENT SETUP AND INFRASTRUCTURE

A. Initial Development Environment and Early Challenges

The development environment was established on Ubuntu 18.04 LTS using Python 3.10 for the agent implementation. Android Debug Bridge (ADB) provided the interface for device communication and control. Google’s Agent Development Kit (ADK) served as the foundation for LLM integration, offering structured abstractions for agent composition and tool invocation. Initial development encountered challenges with emulator stability and timing inconsistencies. These issues motivated a transition to physical device testing, which provided more reliable conditions for agent evaluation. The Android emulator’s simulation of touch input and UI rendering introduced variability that complicated assessment of agent decision-making quality.

B. Framework Integration Challenges and Architectural Limitations

The system integrates Google’s Gemini model family through the ADK framework. Several technical considerations emerged during integration. Session lifecycle management required careful attention to ensure agent state consistency across multiple planning cycles. JSON parsing strategies were implemented to reliably extract structured actions from LLM outputs, including fallback mechanisms for handling partial or malformed responses. API rate limiting and quota management influenced the system’s operational characteristics, particularly during intensive testing phases requiring frequent model invocations. The implementation includes exponential backoff strategies and request batching to work within these constraints while maintaining reasonable test execution times.

C. Strategic Migration to Physical Device Testing

Testing migrated to a physical Google Pixel 6 Pro (1440×3120 resolution) to provide consistent, reproducible conditions for agent evaluation. Physical hardware eliminated emulator-related variability in touch response, rendering behavior, and timing characteristics. This configuration enabled more accurate assessment of agent planning decisions by removing infrastructure-related confounding factors. The physical device setup provided authentic UI rendering, genuine touch sensor responses, and realistic performance constraints. Direct USB connection ensured stable ADB communication with minimal

latency. This environment closely approximates real-world deployment conditions, making experimental results more indicative of practical system behavior.

III. FRAMEWORK ANALYSIS

Google’s Agent Development Kit (ADK) was selected for its native support of LLM-powered agent patterns and clean architectural separation of planning and execution concerns. The framework provides modular abstractions that facilitate composition of specialized agents, each responsible for distinct aspects of the testing workflow. ADK’s event-driven design accommodates the asynchronous nature of LLM API calls while maintaining responsive system behavior. The framework’s built-in support for tool calling enables seamless integration of ADB commands as callable functions within the agent’s action vocabulary. This architecture allows agents to reason about high-level test objectives while accessing low-level device control capabilities. The framework supports structured output formatting, which proved essential for reliably extracting action specifications from LLM responses. Type definitions for action schemas enable validation of generated plans before execution. The framework’s extensibility allowed customization of agent behaviors through specialized prompting strategies and custom tool implementations. Experimentation with different Gemini model variants (Flash and Pro) revealed trade-offs between response latency and reasoning depth. Flash variants provided faster iteration cycles suitable for rapid action selection, while Pro variants demonstrated more sophisticated understanding of complex UI states. The architecture’s model-agnostic design facilitated this comparative evaluation.

IV. MULTI AGENT SYSTEM ARCHITECTURE

The system implements a three-tier multi-agent architecture that distributes responsibilities across specialized agents. This design separates strategic planning from operational execution and oversight, enabling each component to focus on its core competency while maintaining clear interfaces between layers.

A. Planner Agent: Strategic Action Selection

The Planner Agent serves as the system’s reasoning engine, analyzing current UI state and selecting appropriate actions to advance toward test objectives. It receives compressed state representations extracted from UI hierarchy dumps, recent action history for temporal context, and high-level goal descriptions. The agent outputs structured action specifications including operation type, target coordinates, and confidence assessments. The planner operates within a constrained action vocabulary: TAP for element interaction, TYPE for text entry, SWIPE for navigation gestures, WAIT for timing control, and VERIFY for assertion checking. This bounded action space helps the LLM generate executable plans while providing sufficient expressiveness for complex workflows. Prompt engineering strategies guide the planner toward effective action selection. The prompts emphasize forward progress through test steps, provide explicit state transition expectations, and include examples of successful action sequences for similar scenarios. Context window management ensures the agent retains relevant history without exceeding token limits.

B. Executor Agent: Deterministic Action Implementation

The Executor Agent translates high-level action specifications into concrete ADB commands. It maintains responsibility for precise coordinate calculation, touch event generation, text input handling, and timing control. The executor validates action preconditions through UI XML inspection before attempting execution, enabling early detection of infeasible actions. Action execution follows a verified workflow: capture pre-execution screenshot, validate target elements exist, execute ADB command with appropriate parameters, wait for state transition completion, and capture post-execution screenshot. This comprehensive capture strategy provides detailed execution traces useful for debugging and analysis. The executor implements retry logic for handling transient failures such as temporary UI unresponsiveness or animation-induced timing issues. Exponential backoff strategies prevent overwhelming the system during failure cascades. Screenshot comparison and UI hierarchy validation provide multi-modal confirmation of successful action completion.

C. Supervisor Agent: Global State Management and Verification

The Supervisor Agent maintains global test state and coordinates the overall workflow. It tracks visited UI states through fingerprinting, enabling detection of cyclic behavior patterns. The supervisor enforces action constraints to prevent potentially destructive operations and maintains progress metrics to assess test advancement. Loop detection employs state hashing to identify when the system revisits previously encountered configurations. Upon detecting cycles, the supervisor can intervene by constraining the planner’s action space, suggesting alternative approaches, or triggering predetermined recovery sequences. This intervention capability helps escape local minima in the action space. Final test outcome determination considers multiple factors: completion of all required steps, achievement of expected final state, execution time within acceptable bounds, and absence of critical errors during workflow execution. This multi-criteria evaluation provides nuanced assessment beyond simple pass/fail binary outcomes.

D. Inter-Agent Communication and Coordination

Agent coordination follows a hierarchical message-passing architecture where the supervisor maintains overall control flow, the planner operates in advisory capacity producing action recommendations, and the executor provides capability guarantees through validated action primitives. All inter-agent messages are versioned and timestamped to facilitate debugging and post-execution analysis. The architecture implements explicit synchronization points at critical phases of test execution, ensuring that all agents reach consistent states before proceeding to the next major phase. This synchronization proved essential for maintaining system-wide coherence when individual agents operated at different speeds due to variable LLM response latency or device interaction timing.

V. EXPERIMENTAL EVALUATION

A. Test Suite Design and Methodology

The evaluation suite comprises four test cases designed to assess agent capabilities across different scenarios. Tests were structured to evaluate both positive functionality (successful workflow completion) and negative validation (correct failure identification).

Table I
FOUR QA TESTS WERE IMPLEMENTED:

Test	Description	Expected Outcome
Test 1	App onboarding and vault creation	PASS
Test 2	Note creation and return to main UI	PASS
Test 3	Invalid settings navigation	FAIL
Test 4	Unsupported feature access	FAIL

Test 1 examines the agent’s ability to navigate complex onboarding sequences involving multiple screens, decision points, and permission handling. The workflow requires 10 discrete steps with state-dependent branching. Test 2 evaluates content creation and manipulation, testing the agent’s capacity for text input and UI state management. Tests 3 and 4 assess negative validation capabilities, verifying that the system correctly identifies invalid states and missing features. These tests confirm the supervisor’s failure detection mechanisms operate as intended and that agents can distinguish between expected success and intentional failure scenarios.

B. Quantative Results and Performance Metrics

The LLM-based planning agent demonstrated sophisticated understanding of UI context and goal-oriented reasoning. In successful executions, the agent effectively parsed UI hierarchies, identified relevant interaction targets, and generated appropriate action sequences. The system successfully navigated complex multi-step workflows when provided with clear state information and well-structured objectives. Agent behavior revealed interesting patterns in action selection. The planner showed preference for familiar navigation patterns learned from training data, occasionally selecting common actions (such as BACK or HOME) even when alternative paths would be more direct. This behavior suggests strong pattern matching capabilities but also indicates opportunities for improved context-specific reasoning. Execution timing varied significantly based on workflow complexity and API response latency. Simple linear workflows completed efficiently, while complex decision trees with multiple state assessments required extended execution times. The system demonstrated ability to recover from transient errors through retry mechanisms, though this added to overall execution duration. The

supervisor’s intervention mechanisms proved valuable for preventing unproductive exploration of the action space. Loop detection successfully identified circular navigation patterns, enabling corrective actions. Multi-criteria outcome assessment provided nuanced evaluation of test results, distinguishing between different failure modes and partial successes.

VI. DEMONSTRATION AND REPRODUCIBILITY

A. Video Documentation Methodology

Complete system operation was documented through video recording to provide transparent demonstration of agent behavior. The recording utilized SimpleScreenRecorder on the Linux development workstation, configured for high-quality capture with minimal system impact. The Android device’s screen was shared through Zoom’s screen sharing capability, enabling real-time visualization while maintaining stable connection quality. This recording approach balanced multiple objectives: authentic representation of agent behavior on physical hardware, clear visualization of UI interactions and state transitions, acceptable recording quality for analysis purposes, and minimal interference with system performance during test execution.

B. Repository and Resources

The complete demonstration video and system implementation are publicly available to enable reproducibility and extension of this work. The repository includes:

- Repo url: https://github.com/ssarka4894/mobile_qa_agent/tree/main
- Video url: https://github.com/ssarka4894/mobile_qa_agent/blob/main/Qualgent_QA_Agent_Automation_Demo-2025-12-18_20.20.12

The repository contains complete source code for all three agents, test case definitions with action specifications, configuration files for ADK and Gemini integration, setup documentation for reproducing the experimental environment, and execution logs from experimental trials. This comprehensive resource package enables other researchers to validate findings, explore alternative approaches, and build upon this architecture.

C. Observable Agent Behaviors

The video demonstration reveals several characteristic agent behaviors. Viewers can observe the planner’s reasoning process through action selection patterns, the executor’s precise coordinate-based interactions, the supervisor’s intervention during loop detection, and the system’s handling of both successful workflows and expected failure conditions. The recording provides visual evidence supporting the quantitative metrics and behavioral observations discussed in this paper.

VII. DISCUSSION

This research demonstrates the current capabilities and challenges of applying LLM-based agents to mobile test automation. The agents successfully executed complex workflows

when provided with appropriate context and clear objectives, showcasing the potential of AI-driven testing systems. The multi-agent architecture proved effective at distributing specialized responsibilities while maintaining coordination. Several factors influence agent effectiveness in this domain. State representation quality significantly impacts planning decisions; agents perform better with structured UI hierarchies than with raw visual information alone. The probabilistic nature of LLM outputs introduces variability in agent behavior. While this can enable adaptive responses to unexpected situations, it also complicates reproducibility and consistency requirements typical in testing contexts. The system benefits from mechanisms that constrain this variability where appropriate, such as action validation and state transition verification. LLM agents show particular promise for several testing applications: generating test cases from natural language requirements, reasoning about complex assertion logic, analyzing failure patterns in execution logs, and adapting to UI changes through flexible understanding. These capabilities complement traditional automation approaches by adding cognitive reasoning where rigid scripts struggle. Future research directions include enhanced visual grounding techniques for improved UI understanding, explicit state machine constraints to guide agent behavior, formal verification methods for action sequence validation, and ensemble approaches combining multiple agent strategies. Integration with existing testing frameworks could leverage agent capabilities while maintaining compatibility with established workflows.

VIII. CONCLUSIONS AND FUTURE DIRECTIONS

This work presents a complete implementation of an LLM-based multi-agent system for mobile QA automation, demonstrating both capabilities and limitations of current approaches. The three-tier agent architecture successfully coordinates planning, execution, and supervision across complex testing workflows. Experimental evaluation on physical hardware provides realistic assessment of agent behavior in authentic deployment conditions. The research reveals that LLM agents can effectively understand testing objectives, reason about UI state, and generate appropriate action sequences for many scenarios. The agents demonstrated sophisticated behavior including multi-step planning, context-aware decision making, and successful navigation of complex application workflows. These capabilities suggest significant potential for AI-augmented testing systems. Practical deployment of such systems requires careful consideration of their characteristics. Agent behavior exhibits variability inherent to probabilistic models, necessitating robust validation and monitoring mechanisms. Performance considerations include API latency and token processing costs. Effective prompt engineering and action space design significantly influence system capability and reliability. The architecture presented provides a foundation for building sophisticated AI-driven testing tools. By separating planning, execution, and supervision concerns, the design enables focused optimization of each component. The open-source implementation and comprehensive documentation facilitate further research and practical applications of these techniques. As LLM capabilities

continue advancing, agent-based testing systems will become increasingly capable. This research contributes understanding of how to effectively structure and deploy such systems, providing architectural patterns and empirical insights that inform future development. The combination of intelligent reasoning with reliable execution mechanisms offers a promising direction for next-generation mobile testing automation.

IX. ACKNOWLEDGEMENTS

I am deeply grateful to the QualGent research team for providing this opportunity to explore the intersection of large language models and mobile quality assurance automation. The task and associated test cases have been extraordinarily interesting and challenging, offering profound insights into both the capabilities and design considerations of agentic AI systems. Irrespective of the outcome of this challenge, this has been a tremendously rewarding experience that has significantly deepened my understanding of practical AI system development and deployment. This project represented my first experience integrating large language models with Android automation frameworks, providing an invaluable learning opportunity across multiple dimensions: multi-agent system architecture and coordination, LLM prompt engineering for structured outputs and reliable action generation, integration strategies for combining AI reasoning with system-level controls, and production-grade error handling and recovery mechanisms. The challenges encountered during development from framework integration complexities to managing LLM behavior variability; proved to be excellent learning experiences that will inform all future work in this rapidly evolving domain. Special appreciation to the open-source community maintaining the tools and frameworks that made this research possible, including the Android Open Source Project, Google's Agent Development Kit team, and the countless contributors to the Python automation ecosystem. Their dedication to building and sharing robust tools enables research and innovation throughout the field.

REFERENCES

- [1] Google, "Google/adk-docs: An open-source, code-first toolkit for building, evaluating, and deploying sophisticated ai agents with flexibility and control." [Online]. Available: <https://github.com/google/adk-docs>
- [2] "Android developers." [Online]. Available: <https://developer.android.com/tools/adb>
- [3] OpenAI and J. A. et. al, "Gpt-4 technical report," 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [4] J. Wang, H. Xu, J. Ye, M. Yan, W. Shen, J. Zhang, F. Huang, and J. Sang, "Mobile-agent: Autonomous multi-modal mobile device agent with visual perception," *arXiv preprint arXiv:2401.16158*, 2024.
- [5] C. Rawles, A. Li, D. Rodriguez, O. Riva, and T. Lillicrap, "Android in the wild: A large-scale dataset for android device control," 2023. [Online]. Available: <https://arxiv.org/abs/2307.10088>
- [6] G. T. et. al, "Gemini: A family of highly capable multimodal models," 2025. [Online]. Available: <https://arxiv.org/abs/2312.11805>
- [7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2201.11903>